ReasoningFlow with Tool Calls and Web Search

Streaming Graph Design and Developer Specification | Proposed Extension v0.1 | 11 October 2026

Nathan Conklin
nathan.conklin@vt.edu
Department of Computer Science,
Virginia Tech

Abstract

This design extends ReasoningFlow to represent exposed reasoning text, tool execution, and evidence provenance in one inspectable graph. It retains the paper's eight reasoning-node categories, its Context category, and all 14 semantic edge labels; it adds ToolCall and ToolResult nodes and six operational or provenance relationships. The resulting profile contains 11 node types and 20 edge types. The application records provider and orchestration events as they arrive, constructs the execution portion from recorded identifiers, and annotates complete reasoning units asynchronously. A final reconciliation pass can revise segmentation, classifications, and inferred relationships. Expensive annotation starts when the user opens the reasoning visualization; the underlying event capture continues while the view is closed. The approved legend specifies rectangular nodes, colored label boxes, and separate line styles for original, operational, and provenance relationships. This document defines the representation, streaming lifecycle, developer contracts, visual behavior, failure handling, and evaluation plan. The extension is a proposed design from this discussion, not a capability evaluated in the ReasoningFlow paper. Its semantic edges describe the expressed reasoning trace; they do not establish the model's hidden computational dependencies. [1, §§3–4, 8; 2]

1. Purpose, scope, and decision baseline

1.1 Purpose

The developer should build a replayable reasoning-and-execution graph for a chat interaction. A user should be able to inspect the model's expressed plan, the tool calls that followed, the information returned, and the later statements associated with that information. The same representation should support a live view during generation and a cached view after the interaction ends.

The immediate application is the reasoning-tree tab discussed in this chat. The underlying data structure is a directed acyclic graph, or DAG; a tree view is an optional projection. A step may have several premises, a plan may lead to several operations, and a returned document may inform several later statements. Reducing these relationships to a single-parent tree would discard information. [1, §3]

1.2 Status and scope of this specification

The schema and visual choices in Table 1 capture the latest accepted design. More detailed runtime contracts in later sections are proposed implementation choices that make that design executable. They are not claims about an existing implementation, measured performance, or an API's currently supported fields.

Decision	Baseline for implementation
Original schema	Preserve all nine displayed ReasoningFlow node categories, including Context, and all 14 original semantic edge labels.
New node types	Add exactly ToolCall and ToolResult. Web search is a ToolCall subtype.
New edge labels	Add invokes, returns, provides, `grounded_in`, cites, and `retry_of`.
Node shape	Use the same rectangular or rounded-rectangular box treatment for every node, including ToolCall and ToolResult.
Label presentation	Put semantic edge names inside colored boxes; retain colored arrow lines.
Legend	Use the approved image headed “Legend” and “ReasoningFlow + Tools.”
Computation	Support incremental annotation with final reconciliation; do not block the primary chat stream.
Lazy activation	Capture raw events continuously. Defer expensive semantic annotation until the reasoning view is opened.
Meaning of the graph	Separate recorded execution links from inferred intent and evidence-use links.

Table 1. Accepted design baseline and implementation scope. Source: this discussion and the final approved legend. [2]

Version 0.1 is an observational visualization and annotation system. It does not execute tools because an annotation says they should run. It does not expose hidden model state, reconstruct missing private reasoning, or certify the correctness of an answer. Optional editing of annotations changes the graph interpretation; it does not silently rewrite the original conversation or restart the agent.

1.3 Source boundaries

ReasoningFlow supplies the original annotation vocabulary, the earlier-to-later DAG constraint, the automatic annotation pipeline, and the empirical results cited here. The discussion supplies the two operational nodes, six added links, lazy streaming architecture, and approved visual design. Data contracts, queue behavior, and acceptance tests below are proposed engineering elaborations. They are identified as such where a developer choice is still required. [1; 2]

The paper's Appendix A contains node definitions in Table 3 and edge definitions in Table 4. The earlier reference in the chat to semantic edges in Table 3 should be read with that distinction. The companion document, ReasoningFlowWithToolCallAnnotationGuide.docx, extends Appendix A rather than replacing it.

2. Approved legend and visual language

The approved legend is the visual baseline. It includes all 11 node types and 20 edge types. The two new nodes remain boxes, and each semantic relationship name appears inside a colored label box. “Retrieval,” “Computation,” and “External action” are subtypes of ToolCall, not additional top-level node categories.

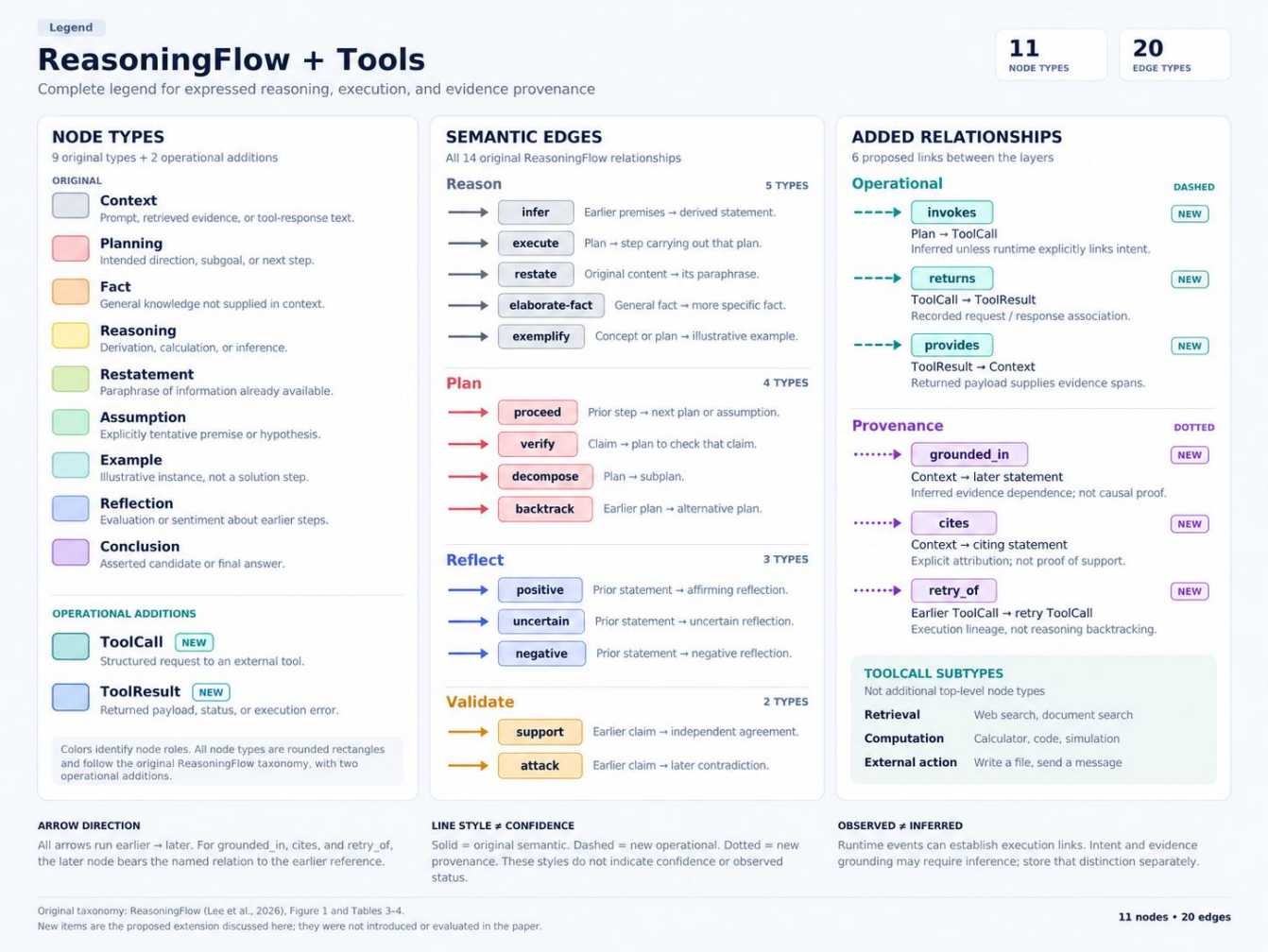


Figure 1: Approved complete legend. Retained unchanged from the final image approved in this discussion. All node types are boxes; semantic edge names are boxed and colored. The picture distinguishes original semantic links, added operational links, and added provenance links. (Source 2)

2.1 Color and line conventions

Node fill indicates role. Original semantic edge families use gray for Reason, pink/red for Plan, blue for Reflect, and orange for Validate. Added operational edges use teal dashed lines; added provenance edges use purple dotted lines, matching the approved image. The earlier general acceptance of dashed added links is realized by these two separate patterns in the approved legend. The application should not silently convert the final dotted provenance lines to a different convention.

Line style describes relationship family, not certainty. A dashed returns link can be backed by an exact recorded call identifier. A solid infer link can be an annotator's interpretation. Store evidence method and review status separately; show them as text or badges in the inspector. A provisional label should have a visible “Provisional” badge rather than a different node shape or an overloaded edge line style.

The approved image is a visual guide, not a machine-readable schema. The developer should implement the label registry in Section 4 and the companion annotation guide. The original paper uses slightly different pastel shades across individual labels; the appendix-style guide preserves those label-level colors. The new operational and provenance colors are design additions, not colors defined by Lee et al. [1, Fig. 1; Appendix A; 2]

2.2 Figure selection and provenance

This document retains the approved legend and relevant original ReasoningFlow illustrations. It also renders the two architecture concepts described in the discussion as readable engineering figures. Earlier generated legends that introduced unsupported categories, omitted Fact, or used nonrectangular operational nodes are superseded and are not included. The figures in this document illustrate the design; they are not experimental results.

3. ReasoningFlow baseline and streaming implications

3.1 Original representation

ReasoningFlow segments a trace into contiguous, nonoverlapping text units, usually sentences or clauses with distinct functional roles. Its eight reasoning-node types are Planning, Fact, Reasoning, Restatement, Assumption, Example, Reflection, and Conclusion. Context represents user-provided material, including retrieved documents and tool responses. The paper's figures consequently display nine categories. Edges connect earlier units to later units and describe logical inference, planning, reflection, or validation. [1, §3; Appendix A, Tables 3–4]

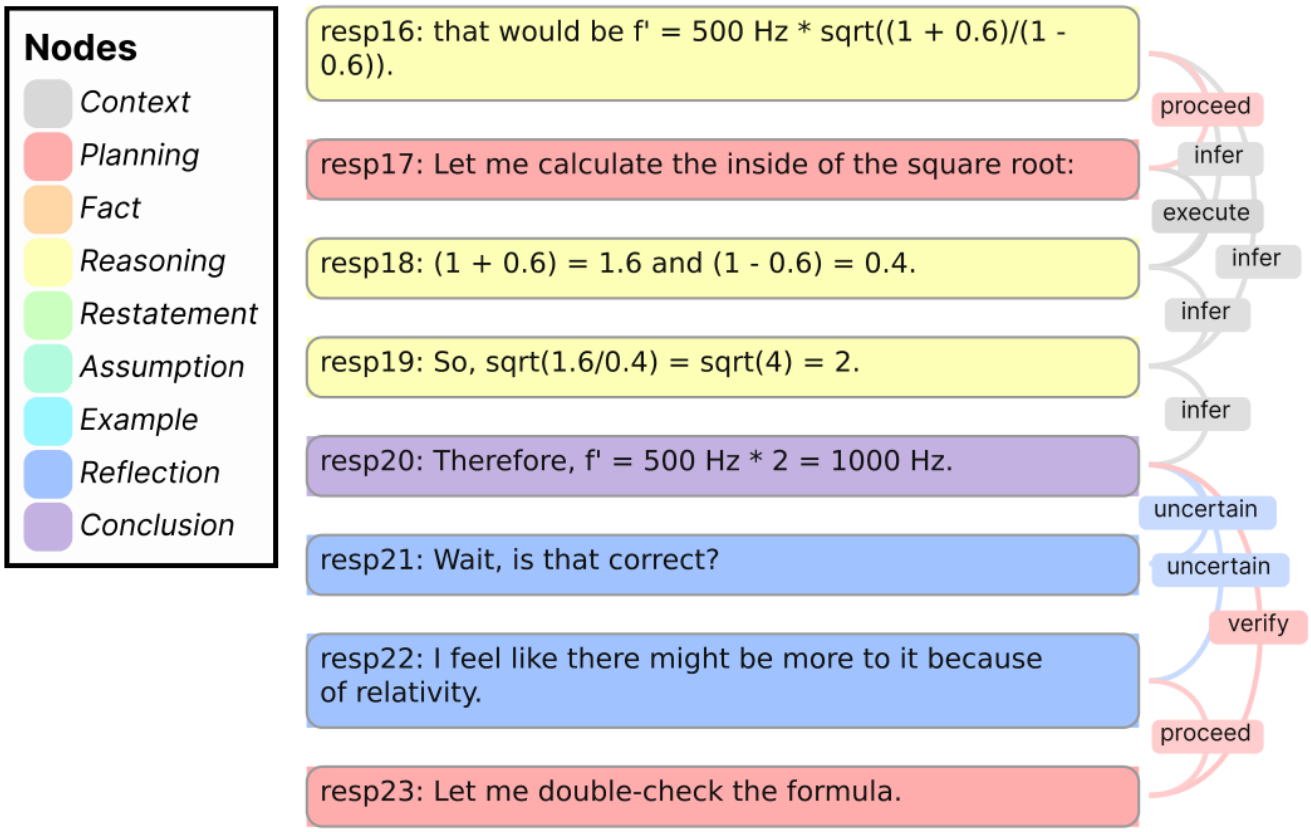


Figure 2: Original ReasoningFlow Figure 1, cropped from the supplied paper. The example preserves chronological text order while displaying deductions, verification plans, and uncertainty. Reproduced from Lee et al. (Source 1, p. 1).

3.2 Automatic annotation pipeline

The paper's automatic pipeline has three main stages, with a separate conclusion-label update inside classification. First, rule-based paragraph segmentation creates manageable text chunks; the authors then use an LLM to divide those chunks into atomic units. Their paragraph rule checks punctuation and the next paragraph's initial letter rather than treating every blank line as a boundary. They use string alignment when returned segments do not match the original text exactly. [1, §4.2.2; Appendix C.1]

Second, the authors classify all nodes together. They initially omit Conclusion and then run a post-hoc step that identifies conclusion nodes. Third, they ask an LLM to find the minimal complete set of predecessors and corresponding labels for each destination node. The allowed incoming edge labels depend on the destination's type. This destination-oriented edge annotation is the part most directly adaptable to streaming. [1, Appendix C.1; Appendix I]

Stage	Paper implementation	Proposed live adaptation
Segmentation	Rule-based coarse boundaries plus LLM refinement and alignment	Buffer the unfinished tail; refine only stable chunks and preserve source offsets.
Node classification	Joint classification of the available complete trace	Classify completed microbatches with prior context; keep labels revisable.
Conclusion update	Separate post-hoc classification	Mark candidate answers during generation; reconcile at episode end.
Edge annotation	Select incoming predecessors for each destination	Query only earlier eligible nodes for each new or revised destination.
Global analyses	Analyze completed graphs	Show current estimates during streaming and reserve final-path summaries for a closed episode.

Table 2. Source algorithm versus the proposed incremental adaptation. The right column is a design proposal, not an evaluated method from the paper. [1, Appendix C.1; 2]

3.3 What can be computed before completion

For a completed new node, earlier candidate sources already exist. The annotator can therefore propose infer, execute, restate, planning, reflection, and validation links without waiting for the complete answer. Long-range links still require access to older candidates. New text may create additional outgoing edges from an older node; the earlier statement in the discussion that an existing step never needs a future-facing edge should not be implemented literally. The restriction is that a newly annotated destination only accepts earlier sources.

The graph is revisable rather than strictly append-only. Later context may change a node's interpretation, expose a correction, or show that an apparent conclusion was an intermediate attempt. Completed graph analyses can also change without any earlier edge being wrong. A branch that appears unused now may become relevant later.

The paper evaluates offline annotations; it does not report streaming accuracy, update latency, or graph stability. Table 5 reports node-classification F1 of 0.865 and edge detection/classification F1 of 0.646 for the best respective annotators using human segmentation. Those numbers are a reference for evaluation design, not expected accuracy for this implementation. The paper also reports higher scores for a separate acceptability review; those are not equivalent to from-scratch human annotation. [1, Appendix C.2–C.4]

3.4 Why execution needs its own representation

A Planning node can express an intention to search. A Context node can contain retrieved text. Neither records the structured operation between them. The paper's execute relationship links a plan to a reasoning step that implements it; it is not a record

of an external API dispatch. ToolCall and ToolResult fill this observational gap while preserving the original semantic vocabulary. [1, Appendix A, Tables 3–4]

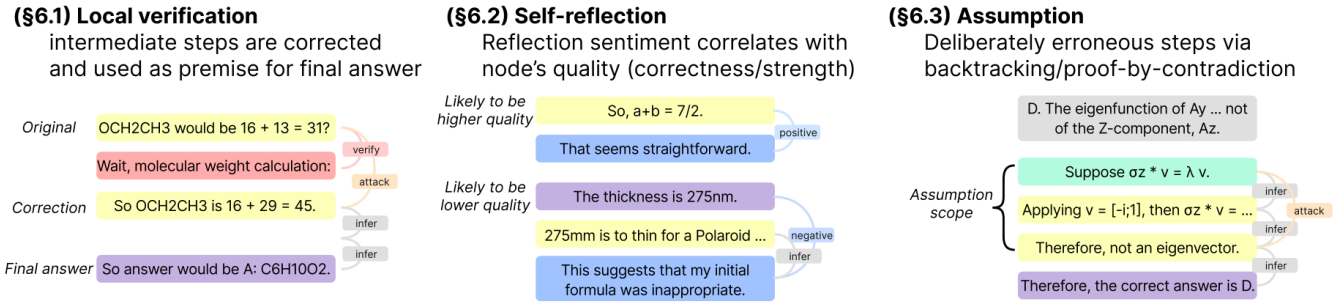


Figure 4: Examples of three fine-grained reasoning behaviors (local verification, self-reflection, and assumption).

Figure 3: Original ReasoningFlow Figure 4, cropped from the supplied paper. Local verification, self-reflection, and assumption-based reasoning remain meaningful when the premises originate in tool results. Reproduced from Lee et al. (Source 1, p. 6).

4. Graph schema and relationship semantics

4.1 Graph layers

Use one typed directed multigraph with three logical layers: semantic reasoning, operational execution, and provenance. Separate views can filter these layers without changing the stored graph. A pair of nodes can have more than one relationship; for example, Context may restate into a later Restatement node and also ground that node. Each edge is independently typed and auditable.

Text-role nodes are spans of an immutable source message or evidence payload. Operational nodes are records of structured events, not sentence fragments. A ToolResult envelope and its extracted Context spans may refer to the same bytes at different representational levels. This is an explicit extension of the original text-only segmentation model, not an additional split of the reasoning text.

4.2 Complete node registry

Label	Meaning and classification boundary
Context	User input, retrieved material, or tool-response text made available as context. It is not a claim that the content is true.
Planning	An expressed direction, subgoal, or proposed next step. A plan is not evidence that execution occurred.
Fact	General or parametric knowledge not already supplied by the context or earlier nodes. A tool result must not be relabeled Fact merely because it appears authoritative.
Reasoning	A derivation, calculation, or inference using earlier information.
Restatement	A copy or paraphrase of earlier content without a new substantive inference.
Assumption	An explicitly tentative premise or hypothesis used within a reasoning scope.
Example	An illustrative instance of a prior concept; a step directly solving the task is generally Reasoning instead.
Reflection	An evaluation, sentiment, or uncertainty statement about earlier content.

Label	Meaning and classification boundary
Conclusion	An asserted candidate or final answer to the task, including an intermediate attempt.
ToolCall	A structured request to an external tool or execution facility. It has its own request and lifecycle metadata.
ToolResult	A returned payload, status, acknowledgment, or error associated with a recorded call.

Table 3. Complete node registry. The first nine labels follow the paper; the final two are the proposed additions. Detailed original definitions remain in Appendix A of [1].

ToolCall has the stable subtype families retrieval, computation, and `external_action`. Retrieval can be refined with an operation name such as `web_search`, `document_fetch`, `file_search`, or `database_query`. A web page or retrieved source is Context. A generated file is an artifact reference in ToolResult and, when its content is exposed to the model, a source of Context. Do not add WebSearch, RetrievedDocument, ExecutionArtifact, Elaboration, or Hypothesis as top-level node labels in this version.

4.3 Original semantic edge registry

Family	Label	Directed reading in the graph
Reason	infer	Earlier premise → later derived statement. Use the original source rather than a duplicate restatement where the guide requires it.
Reason	execute	Earlier plan → later text step implementing the plan. External invocation uses invokes instead.
Reason	restate	Original earlier content → later paraphrase. Preserve the earliest original source when appropriate.
Reason	elaborate-fact	Earlier general fact → later, more specific fact introduced in the same context.
Reason	exemplify	Earlier concept or plan → later illustrative example.
Plan	proceed	Earlier motivating step → later Planning or Assumption node. Apply the more specific planning labels when warranted.
Plan	verify	Earlier claim or reasoning-like step → later plan to check that step.
Plan	decompose	Earlier plan or assumption → later subplan or subassumption.
Plan	backtrack	Earlier plan → later alternative plan; the paper also analyzes assumption-branch patterns.
Reflect	positive	Earlier statement → later affirming Reflection.
Reflect	uncertain	Earlier statement → later uncertain Reflection.
Reflect	negative	Earlier statement → later negative Reflection. An explicit propositional correction may instead require attack.
Validate	support	Earlier proposition → later independently derived, consistent proposition. This is not a generic source-citation link.
Validate	attack	Earlier proposition or assumption → later correction, contradiction, or refutation.

Table 4. Original semantic vocabulary, summarized from the paper's Appendix A, Table 4, and the assumption patterns in Table 9. [1]

Use the paper's permitted source/destination combinations for these labels. Do not silently broaden execute to ToolCall or use proceed as an arbitrary timeline connector. A chronological layout does not require an edge between every adjacent item. Conclusion nodes answering the same question receive support or attack links when the paper's consistency criteria apply; questions must be scoped before comparing conclusions across chat turns. [1, Appendix A]

4.4 Added relationship registry

Label	Stored source → target	Meaning and evidence requirement
invokes	Planning → ToolCall	The call operationalizes the expressed plan. Infer the intent link from text and arguments unless runtime metadata explicitly identifies the plan.
returns	ToolCall → ToolResult	A recorded request/result association established through call and attempt identifiers. It does not imply success.
provides	ToolResult → Context	The returned payload supplies an exact evidence span or a traceable extracted representation. It does not imply the model used the span.
`grounded_in`	Context → later statement	The later statement is attributed to information in the earlier Context. Require a recoverable evidence span and a documented annotation basis.
cites	Context → later citing statement	The later statement explicitly references that context/source. Citation is not proof that the source supports the statement.
`retry_of`	Earlier ToolCall → later ToolCall	The later call retries the earlier operation for the same objective. Prefer explicit attempt lineage over inference from similar arguments.

Table 5. Proposed operational and provenance relationships. The companion annotation guide provides subtypes, examples, exclusions, and adjudication rules. [2]

4.5 Arrow direction and linguistic direction

All drawn edges follow earlier-to-later logical order, as in ReasoningFlow. Three added label names read in the reverse linguistic direction: for Context → Statement labeled `grounded_in`, the statement is grounded in the context; for cites, the statement cites the context; for EarlierCall → LaterCall labeled `retry_of`, the later call is a retry of the earlier call. Document this rule in serialization and tooltips. A generic subject-predicate-object export must invert the subject and object for these three labels.

Event receipt time does not always establish logical order. A provider may deliver citation metadata after the answer, and parallel tool responses may arrive out of order. Keep recorded timestamps, provider sequence information, and a logical/topological ordering separately. A Context node must have been available to the relevant model invocation before a semantic dependence is asserted. If this cannot be established, retain an unresolved reference or an annotation candidate outside the committed graph.

4.6 Core invariants

The graph validator shall reject unknown labels, missing endpoints, self-loops, and cycles. It shall enforce the endpoint matrix and keep operational call IDs consistent. It shall require source spans to remain inside their recorded source representation and require inferred links to cite their annotation inputs. A fixed graph version shall be reproducible from its stored events, source versions, schema version, and recorded annotations.

The validator shall not require a predecessor for every node. A tool can run without any exposed Planning node. Context and Fact can be disconnected. Missing observation is preferable to an invented plan, result, or dependency. The validator shall also distinguish an invalid annotation from an invalid tool execution; a failed annotation must not stop the original chat or tool workflow.

5. System architecture and ingestion

5.1 Runtime components

Use a provider adapter, an append-only event store, an execution projector, a semantic annotation worker, a graph store, and the visualization client. The provider adapter normalizes whichever structured events the chosen endpoint exposes. The execution projector creates ToolCall and ToolResult records without requiring an LLM. The semantic worker performs segmentation, classification, and relationship annotation independently of primary response delivery.

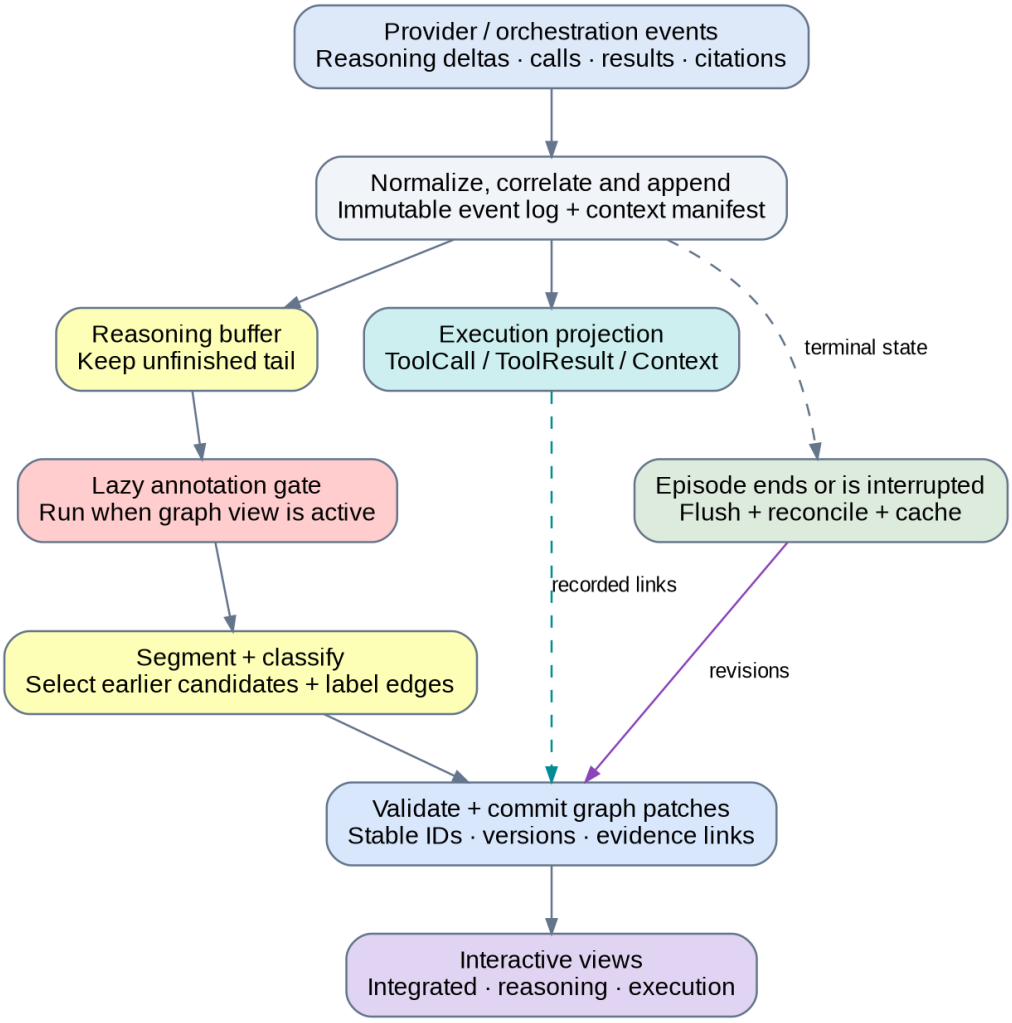


Figure 4: Streaming architecture. The original conversation continues independently of the annotation path. Raw capture and execution projection continue while the graph view is closed; the lazy gate controls expensive semantic annotation. Proposed architecture derived from this discussion. (Source 2)

These component boundaries are a proposed implementation decomposition. They can run as modules in a single application for the prototype. A separate queue service or graph database is not required to preserve the interfaces.

5.2 Normalized event envelope

The adapter should produce an event envelope with a unique event ID, conversation ID, branch ID, episode ID, turn ID, response ID, channel, event type, provider sequence when available, and receive timestamp. Tool events also carry a tool-call ID and attempt ID. Preserve the original provider payload or an access-controlled reference to it.

Normalize the following event families: `reasoning_delta`, `answer_delta`, `tool_call_delta`, `tool_call_ready`, `tool_dispatched`, `tool_result_delta`, `tool_result_ready`, `citation_metadata`, `response_terminal`, and `episode_terminal`. These names are internal design contracts, not a claim that a provider uses those names. Missing families must be recorded as unavailable capabilities rather than synthesized as observed events.

A provider adapter shall distinguish exposed reasoning text, a reasoning summary, and absent reasoning. Record a `disclosure_mode` field. Do not treat a summary as a complete trace or infer unobserved thought nodes to fill gaps. An execution-only graph remains useful when the endpoint returns no usable reasoning text.

[TODO: confirm the target endpoint's event contract, tool identifiers, reasoning disclosure mode, and ordering guarantees before implementing the first provider adapter.]

5.3 Tool request assembly

The adapter may receive tool arguments in fragments. Append them to an argument buffer keyed by call and attempt ID. A ToolCall can appear as a provisional requested operation as soon as its identity is known; its inspector should show “Arguments streaming” until assembly completes. The execution layer, not the annotator, decides when the request is ready to dispatch.

Record request assembly, dispatch, running, and terminal status separately. Observing a structured request does not prove it was dispatched. Observing a dispatch does not prove the external action completed. User cancellation or permission denial can end a call before execution. A denied request still has a record, but its status must not read “Succeeded.”

5.4 Tool results and evidence extraction

Associate results with calls through recorded identifiers, not response order. Preserve structured payloads, errors, status codes, result timestamps, and artifact references. Streaming chunks are updates to a result item, not separate ToolResult nodes for every network packet. An API that returns distinct result items may produce multiple ToolResult nodes for one call; each item has a stable ID.

Create Context nodes for evidence spans intended for inspection or supplied to the model. A search list can provide several snippets. Opening one hit produces a separate ToolCall and ToolResult; the full document is distinct from the earlier search snippet even when both point to the same URL. Evidence metadata must distinguish snippet, full text, extracted text, structured record, and artifact content.

Store extraction offsets or a JSON pointer, the source payload hash, and the extraction method. A transformed representation, such as HTML-to-text, needs its own hash and mapping back to the original result. A provides link records this derivation. It does not establish the source's truth or its actual use in the answer.

5.5 Multi-turn and branch context

Each model invocation needs an input-context manifest describing the messages and evidence actually supplied to it. Earlier conversation content that was truncated, summarized, or excluded must not automatically remain an eligible predecessor for the current invocation. Prior reasoning that was visible to the user but omitted from the next request is not evidence of model access.

Keep turn and response boundaries in the graph. A prior message reused as current context can have a Context occurrence that refers to the original message without duplicating its stored bytes. This is an extension convention for multi-turn use; the paper's dataset does not establish it as a validated chat annotation method. Context reuse and branch provenance remain metadata unless one of the 20 supported edge definitions applies.

Branching or editing a prompt creates a new branch or context revision. Reuse immutable shared sources, but invalidate downstream annotations whose input manifest changed. Do not overwrite the source history that produced the earlier graph.

6. Incremental semantic annotation

6.1 Stable text and the unfinished tail

Store reasoning and answer channels separately. Append text deltas exactly as received, including whitespace and code or mathematical notation. Use a documented character-offset convention. A proposed default is Unicode code-point offsets over the canonical decoded text, with original bytes retained separately; the browser must translate these offsets if its string indexing uses another convention.

The segmenter maintains a stable prefix and an unfinished tail. Token chunks are not reasoning nodes. Sentence punctuation, paragraph boundaries, and closed structured blocks can nominate boundaries, but the segmenter must protect abbreviations, decimals, code fences, lists, and equation blocks. The paper explicitly keeps some multi-clause constructions together, including a conclusion followed by its premise inside one sentence; splitting those constructions can create an artificial backward dependency. [1, Appendix I, Prompt 1]

Use rule-based chunking as a buffer-management step and LLM refinement for functional boundaries. Return offsets or validate returned spans against the original text. Reject rewritten text as a replacement for the source. If alignment fails, keep the raw chunk visible and mark segmentation unresolved. A maximum-buffer threshold may trigger a provisional display chunk, but it should not assert that an incomplete equation is an atomic reasoning step.

6.2 Classification and microbatching

Classify a small batch of stable units using their source text, relevant previous units, task context, and original label definitions. The initial experiment should test batches of three to five completed units, as proposed in the chat; this is a tunable starting point, not a measured optimum. Latency-sensitive implementations can flush a partially filled batch after a configurable delay.

During streaming, Conclusion means an asserted candidate answer, not necessarily the final answer. Record `conclusion_status` separately as candidate or `final_for_episode`. An interrupted trace may contain no Conclusion. Do not copy the paper's dataset-construction requirement to force a conclusion into an unfinished, failed, or execution-only interaction. The final user-visible answer belongs to the answer channel; it can supply Conclusion nodes without being mislabeled as hidden reasoning.

6.3 Candidate predecessor retrieval

For each destination, construct candidates from recent earlier nodes, relevant Context, active plans, active assumptions, earlier candidate conclusions addressing the same question, and long-range semantic matches. Preserve the original sources of restatements. A bounded recent window alone is insufficient for returns to old ideas or distant corrections.

The proposed production path uses retrieval to reduce the annotation prompt. The evaluation path should also support all eligible earlier nodes as an offline baseline. A possible initial comparison is all predecessors versus a recent window of 15 plus a separately budgeted long-range set; this value is an experimental setting from the discussion, not a required semantic cutoff.

Track why each candidate was retrieved. A retrieval score is not an edge probability. The annotator still decides whether a relationship exists and which label applies. Missing a relevant predecessor at retrieval time places an upper bound on downstream edge recall.

6.4 Edge detection and evidence

For each destination, provide the allowed labels and ask for the minimal complete predecessor set. Permit an empty result. A batch request may contain several destinations, but each must have an independent answer and only use sources earlier than that destination. Newly completed units in the same microbatch may serve as predecessors when their order is known.

Each inferred edge should include supporting text spans or recorded event references and a short public annotation justification. This justification explains the label assignment; it is not a request to expose an annotator model's private chain of thought. Do not accept invented node IDs, unsupported relationship types, or references to unavailable context.

The system can commit exact returns and provides links before semantic annotation catches up. invokes is inferred unless a runtime record explicitly maps the call to the displayed plan. cites can be derived from an explicit citation marker, while `grounded_in` requires a separate judgment about information dependence. Do not automatically generate `grounded_in` whenever cites exists.

6.5 Reconciliation and graph revisions

At episode completion, flush stable remaining text, identify final conclusion candidates, and revisit ambiguous boundaries and labels. Relabeling a destination can change its permitted incoming edges; revalidate and reannotate affected destinations. A split or merge must preserve the original spans, retire superseded node versions, and recompute incident edges. Use an atomic revision so the UI never displays edges pointing to missing nodes.

Offer selective reconciliation and a full-trace offline reconciliation mode. Selective reconciliation reduces work but does not guarantee the same graph as a full offline pass. Later text can change the interpretation of older units beyond a small recent window. Store the reconciliation scope in the graph metadata.

Distinguish response completion from episode completion. A response that requests a tool is not necessarily the end of the interaction. Reconcile a closed episode after the orchestration loop reaches a terminal state, or produce an explicitly incomplete snapshot after interruption. A late result may create a later revision, but it cannot be treated as evidence available to an already generated statement unless the recorded ordering supports that interpretation.

6.6 Reference processing loop

```
# Provider-independent pseudocode; the primary stream never waits on annotation.
def on_event(event):
    if event_store.already_seen(event.event_id):
        return
    event_store.append(event)
    execution_projector.apply(event)
    text_buffers.append_if_text(event)
    graph.publish_recorded_patches()

    if view_state.annotation_enabled:
        stable_units = segmenter.take_stable_units(event.episode_id)
        annotation_queue.enqueue_unique(stable_units)

    if event.is_episode_terminal:
        annotation_queue.request_reconciliation(
            episode_id=event.episode_id,
            complete=event.completed_normally
        )

async def annotate_batch(batch):
    snapshot = graph.read_versioned_inputs(batch)
    labels = await classify(snapshot)
    candidates = retrieve_eligible_predecessors(snapshot, labels)
    edges = await label_incoming_edges(snapshot, labels, candidates)
    patch = validator.build_patch(snapshot, labels, edges)
    graph.commit_if_inputs_unchanged(patch)
```

The queue must defer reconciliation until earlier dependent annotation jobs are committed or explicitly superseded. An episode-terminal event received while the tab is closed records that reconciliation is pending; it must not bypass the lazy gate and start an expensive model call.

7. Data model and developer contracts

7.1 Identity and source records

Use stable opaque IDs for events, nodes, edges, sources, model invocations, and graph revisions. Do not use mutable display text as identity. Store the original raw text in source records and reference spans from nodes. A graph revision records `schema_version`, `annotation_version`, `source_revision`, model/input metadata, and the latest consumed event cursor.

A text node's source span and a ToolCall's runtime ID serve different purposes. The common node record should support both without requiring nonexistent text for an execution event. Proposed fields are shown below; runtime implementations may use equivalent typed structures.

```typescript
type NodeType =
  | "Context" | "Planning" | "Fact" | "Reasoning"
  | "Restatement" | "Assumption" | "Example"
  | "Reflection" | "Conclusion" | "ToolCall" | "ToolResult";

type Span = {
  source_id: string;
  representation_hash: string;
  start_codepoint: number;
  end_codepoint: number; // exclusive
};

type GraphNode = {
  id: string;
  type: NodeType;
  conversation_id: string;
  branch_id: string;
  episode_id: string;
  turn_id: string;
  response_id?: string;
  version: number;
  spans: Span[];
  event_ids: string[];
  state: "provisional" | "committed" | "superseded";
  disclosure_mode?: "exposed_text" | "summary" | "none";
  subtype?: string;
  call_id?: string;
  attempt_id?: string;
  result_item_id?: string;
  conclusion_status?: "candidate" | "final_for_episode";
  metadata: Record<string, unknown>;
};
```

A runtime status such as failed or canceled belongs in ToolCall/ToolResult metadata; it is independent of the annotation state. A committed node can represent a failed call. A provisional node can represent an observed request whose arguments have not finished streaming.

7.2 Edge records

```typescript
type EvidenceMethod =
  | "runtime_observed" | "span_derived" | "explicit_reference"
  | "annotator_inferred" | "human_reviewed";

type GraphEdge = {
  id: string;
  source_id: string; // earlier endpoint
  target_id: string; // later endpoint
  label: string;     // must match the 20-label registry
  family: "reason" | "plan" | "reflect" | "validate"
        | "operational" | "provenance";
  version: number;
  evidence_method: EvidenceMethod;
  evidence_spans: Span[];
  event_ids: string[];
  annotation_basis?: string;
  state: "provisional" | "committed" | "superseded";
  annotator_confidence: number | null;
};
```

Confidence is optional and must remain null unless the implementation has a defined scoring method. A self-reported number from an annotator must not be presented as a calibrated probability of correctness. Reflection sentiment, annotation confidence, source credibility, and tool status are separate dimensions.

A canonical active-edge key is `source_id` + `target_id` + label within a graph version. This permits infer and `grounded_in` between the same endpoints without treating one as a duplicate. Provenance metadata should identify a human review without erasing whether the original link was inferred or recorded.

7.3 Logical ordering and manifests

Store the original timestamps and event sequence separately from the layout's topological index. Add required ordering constraints from call/result associations, source availability, and text order, then derive a topological layout. Parallel sibling calls do not gain a semantic relationship merely because the application needs a display order.

The context manifest associates a model invocation with the exact source representations and ranges it received. A candidate selector should enforce this eligibility before similarity search. Replayed historical evidence can retain its original source identity while receiving a new context-occurrence record. The manifest is also the basis for cache invalidation after context edits or summarization.

7.4 Patch and persistence API

Graph updates should be atomic patches carrying `graph_id`, `base_version`, `new_version`, `consumed_event_cursor`, `nodes_upserted`, `edges_upserted`, `node_ids_retired`, and `edge_ids_retired`. A client applies a patch only to the matching base version. On a gap, it requests a snapshot rather than guessing the missing changes.

Proposed service operations are `get_graph_snapshot`, `subscribe_graph_patches`, `activate_annotation`, `pause_annotation`, `request_reconciliation`, and `export_graph`. These can be local function calls for the prototype or API endpoints in a distributed implementation. The main chat transport remains a separate path.

Persist raw events and annotations independently. Replay can reconstruct the recorded execution layer deterministically. Reproducing an inferred layer requires the stored annotation outputs; rerunning a model does not guarantee identical annotations even with identical prompts. Keep the model/configuration identity and source hashes for comparison.

8. Annotation prompts and validation contracts

8.1 Segmenter contract

The segmenter input is a complete eligible chunk plus surrounding boundary context. Its output is an ordered array of nonoverlapping source ranges. It must preserve punctuation and whitespace through the referenced source, cover the intended chunk, and avoid returning rewritten versions of the text. The prompt should include the paper's segmentation distinctions, particularly mixed functional clauses and unsplit code/equation blocks. [1, Appendix I]

8.2 Classifier and relationship contracts

The classifier receives the nine original node labels, definitions, current task, and eligible source text. ToolCall and ToolResult are supplied by the execution projector; the classifier does not hallucinate them from prose. It may label a statement about a tool as Planning, Reasoning, Reflection, or another original role.

The relationship annotator receives the destination, eligible predecessors, and endpoint-specific allowed labels. Ask for `source_node_id`, label, evidence references, and a short annotation basis. An empty incoming-edge list is valid. For the new relationships, apply the definitions in the companion annotation guide rather than a generic “related to” instruction.

Guard	Required response
Unknown node ID or label	Reject the candidate; do not add it to the graph.

Guard	Required response
Source later than destination	Reject or hold pending order resolution.
Source not in the invocation's available context	Do not commit `grounded_in` or infer from that source.
Unsupported intent claim	Keep the call visible without invokes.
Citation without a recoverable source	Preserve unresolved citation metadata; do not fabricate a Context node with invented content.
Rewritten segmentation	Reject the rewritten text and retain the original source buffer.
Low-quality or missing annotation	Show “Unresolved” and allow later review; continue the main interaction.

Table 6. Validation behavior for generated annotations. These are proposed implementation safeguards.

8.3 Separation from execution

Treat reasoning text, retrieved pages, and tool output as untrusted annotation data. The annotation worker shall not follow instructions embedded in those inputs or invoke external actions. Use a fixed system instruction, a typed output schema, and explicit data boundaries. Logging or replaying a ToolCall must never dispatch the call again. The execution subsystem retains its own authorization and confirmation controls.

9. Tool-call and web-search walkthrough

9.1 Integrated flow

A search or document fetch fits the original semantic categories around the new operational records. The plan is Planning, the request is ToolCall, the returned envelope is ToolResult, and the evidence is Context. A literal paraphrase of that evidence is Restatement; a derived claim is Reasoning; an asserted answer is Conclusion. [1, Appendix A; 2]

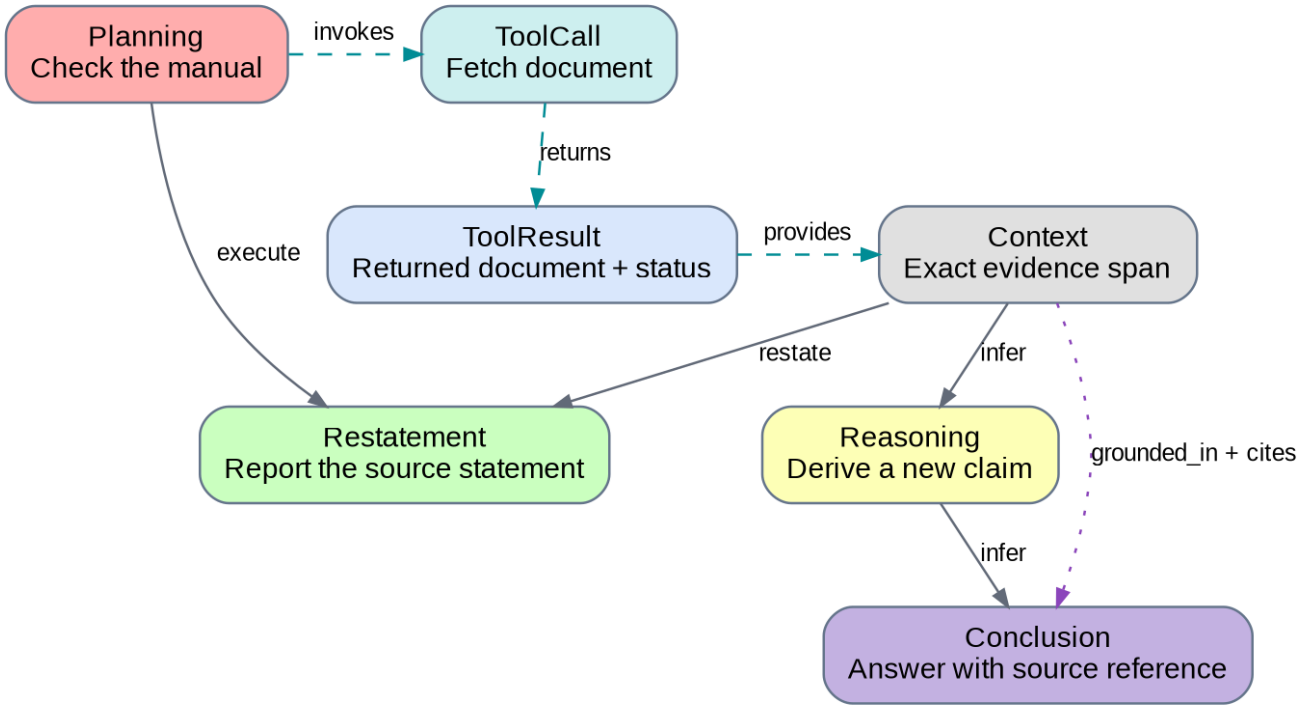


Figure 5: Integrated reasoning, execution, and evidence flow. The figure renders the conceptual graph discussed in the chat with the accepted two operational nodes. Multiple semantic and provenance links can coexist; cites is added only when an explicit source reference exists. Proposed extension. (Source 2)

The diagram's execute link connects a plan to the later text step implementing it. The invokes link records a separate association with the external operation. They are not interchangeable. Similarly, provides means an evidence span comes from a result, `grounded_in` attributes information dependence, and cites records explicit attribution.

9.2 Worked search-and-correction trace

The following example is synthetic. DemoCache and its documentation are invented solely to illustrate annotation. The search snippet says a default timeout is 20 seconds; the opened manual says 30 seconds. The example does not assert that either value describes a real product.

ID	Node type	Recorded event or text
P1	Planning	“I should search for the DemoCache default timeout.”
T1	ToolCall	retrieval/`web_search`; `call_id`=search-1; query=DemoCache default timeout.
U1	ToolResult	Search list containing snippet S1; the result is associated with search-1.
C1	Context	S1: “Default timeout: 20 seconds.” The snippet is supplied to the model.
K1	Conclusion	“The default timeout is 20 seconds.” This is a candidate answer.
F1	Reflection	“That answer may depend on an outdated snippet.”
P2	Planning	“I should check the manual itself.”
T2	ToolCall	retrieval/`document_fetch`; `call_id`=fetch-1; fetch the S1 target.
U2	ToolResult	The document-fetch payload and retrieval status.

ID	Node type	Recorded event or text
C2	Context	S2: “The default timeout is 30 seconds.” Full-document evidence supplied to the model.
R1	Restatement	“The manual specifies a default timeout of 30 seconds.”
K2	Conclusion	“The default timeout is 30 seconds [S2].” Final answer for this episode.

Table 7. Synthetic trace with search, source verification, and correction.

Recorded or extraction-derived links are T1 → U1 returns, U1 → C1 provides, T2 → U2 returns, and U2 → C2 provides. P1 → T1 invokes and P2 → T2 invokes are intent links; they require annotation unless the runtime explicitly maps the plans. C2 → K2 cites is an explicit reference when S2 resolves to that context.

Semantic links can include C1 → K1 infer, K1 → F1 uncertain, K1 → P2 verify, F1 → P2 proceed, C2 → R1 restate, and C2 → K2 infer. The original source C2 is preferred over the duplicate R1 as a premise where the paper's restatement rule applies. K1 → K2 attack captures the contradictory candidate answers. C2 → K2 `grounded_in` can accompany infer and cites when the evidence attribution is warranted.

The statement “The snippet reports 20 seconds” would have been a Restatement and would still accurately describe the snippet. The later manual does not contradict the historical fact that the snippet said 20 seconds. The example therefore makes K1 an asserted answer so that the attack link applies to the proposition actually corrected.

9.3 Technical retry versus reasoning backtracking

Suppose T2 fails with an observed timeout and the orchestrator repeats the same fetch as T3 under a recorded retry policy. T2 → T3 `retry_of` expresses execution lineage. The failure can have its own ToolResult with error metadata. A later plan to search a different source after reconsidering the evidence is a different phenomenon: connect the earlier and later plans with backtrack when the original criteria apply, and connect each plan to its own ToolCall with invokes.

A repeated-looking query is not sufficient evidence of a retry. The model may be verifying a source independently or exploring another hypothesis. Identical arguments can support an annotation candidate, but only recorded lineage or a justified review should commit `retry_of`.

9.4 Parallel, empty, and opaque results

One plan can invoke several parallel retrieval calls. Associate each result with its call ID even if results arrive in reverse order. A synthesis node can have multiple Context predecessors; maintain their independence rather than routing all evidence through the most recent result.

An empty search list is still a ToolResult. It can provide Context stating that this query returned no hits, but it does not prove that no relevant information exists. A partial document is evidence only for its returned portion. A provider-hosted search with opaque internals should show the operation and result scope that the API actually exposes; do not invent hidden subqueries or page opens.

10. Visualization behavior and interaction

10.1 Lazy view lifecycle

State	Required behavior
Tab closed; response streaming	Capture raw deltas and tool events. Maintain cheap recorded execution state. Do not start expensive semantic annotation.

State	Required behavior
Tab opened midstream	Read a high-water event cursor, enqueue the unannotated stable prefix, and subscribe to later events without duplication.
Tab open; annotation behind	Show the raw stream and recorded execution immediately. Display annotation progress and lag separately.
Tab closed after activation	Pause new semantic jobs by default; allow an in-flight job to finish or cancel according to configuration. Preserve its cursor.
Episode ends while tab closed	Record terminal state and pending reconciliation. Defer model-based work until activation.
Episode ends while tab open	Drain prior jobs, reconcile the selected scope, and save a versioned snapshot.
Tab reopened	Load the cached graph and process only missing or invalidated inputs.

Table 8. Lazy initialization and caching behavior from the discussion, made explicit for implementation.

Opening the tab must not lose or double-annotate text emitted during backfill. Use the captured high-water cursor to partition the replay prefix from the live suffix. Idempotent event and annotation-job IDs make repeated opens safe.

10.2 Views and layout

Provide reasoning-only, execution-only, and integrated views. The reasoning view retains the original semantic nodes and selected evidence. The execution view shows calls, results, statuses, and their provenance. The integrated view places them together in chronological/topological order. The user's "reasoning tree" label can remain in the interface, but the backend and full view should preserve DAG joins and crossing links.

Use stable positions for existing nodes during live updates. Add new nodes near their logical position, preserve the current viewport, and make a full re-layout an explicit command. Allow collapse by turn, plan, or tool result; collapsed groups must not silently drop cross-group relationships. Hover or selection should reveal the exact text span or recorded event rather than only a generated summary.

The graph can be dense. Default to readable node boxes, short labels, and detail-on-demand. Give the user filters for node type, relationship family, review status, and invocation. Provide a visible legend toggle. Color must be redundant with printed type names and edge labels.

10.3 Inspectors and evidence status

A node inspector should show role, exact source text, message/channel, logical order, annotation version, and status. ToolCall adds tool name, arguments, attempt lineage, dispatch evidence, and completion state. ToolResult adds payload type, return status, errors, artifacts, and linked Context spans.

An edge inspector should show direction, label definition, source and destination, evidence method, supporting spans or event IDs, and the annotation basis. For `grounded_in`, cites, and `retry_of`, display a complete sentence that reads the inverse-name convention correctly. Show "Inferred" separately from "Recorded" and "Human reviewed."

10.4 Branches, corrections, and uncertainty

Highlight a later correction without erasing the earlier claim. Keep intermediate conclusions visible as prior attempts. Reflection sentiment is a statement expressed by the model, not a calibrated confidence score. The UI should label "uncertain reflection" rather than converting it into an unsupported percentage.

A disconnected branch should be described as “No annotated path to the selected conclusion” rather than “Unused internally.” To compute a reasoning-premise view, select appropriate semantic edges, principally infer with the paper's restatement handling. Do not traverse returns, cites, or attack as if each were a positive logical premise. Otherwise an abandoned wrong answer can appear to support the final answer merely because an attack edge connects them. The paper itself distinguishes discourse structure from mechanistic dependence. [1, §§7–8]

11. Runtime efficiency and scheduling

11.1 Cost model

The paper allows $O(N^2)$ possible earlier-to-later pairs and annotates predecessors per destination. Its observed graphs average approximately 1.3–2.0 edges per node across the configurations reported in Section 5.1. Sparse output does not make full-history prompt processing linear: repeatedly sending a growing prefix can still consume quadratic aggregate input text. [1, §§4.2.2, 5.1]

For 200 nodes, all earlier-node comparisons yield $200 \times 199 / 2 = 19{,}900$ candidate pairs. A recency-only window of 15 yields 2,880 candidate pairs before adding long-range candidates. These are arithmetic workload illustrations from the discussion, not measured runtimes. The bounded strategy trades candidate recall for smaller prompts; the evaluation must report that tradeoff.

11.2 Queue discipline

Use separate resources or priorities for the primary chat and annotation work. The ingestion path should append events and return quickly. Annotation jobs may lag; they must not delay token delivery. Batch stable units, cache unchanged source representations, and deduplicate jobs by destination, source revision, and schema version.

Keep queue length, oldest-job age, annotated-event cursor, and active request count observable. If arrivals exceed annotation throughput, coalesce compatible jobs or defer semantic annotation. Do not drop raw source events to hide the backlog. A retrieval cache can store eligible candidates, but it must be invalidated when the context manifest changes.

11.3 Proposed configuration surface

Setting	Proposed starting behavior
`annotation_mode`	lazy; semantic work begins on view activation.
`microbatch_completed_units`	Evaluate 3–5 units; allow a time-based flush.
`candidate_strategy`	`recent_plus_long_range`, with `all_eligible` available for evaluation.
`reconciliation_scope`	selective for interaction; `full_trace` as an explicit comparison mode.
`inference_concurrency`	Bounded and independent of the primary chat; choose after endpoint testing.
`provenance_visibility`	Show labels and evidence methods; never use line style as confidence.
`event_retention`	Configurable; the deployment owner must approve the storage policy.
`missing_reasoning_behavior`	Render the observed execution/evidence graph and mark reasoning unavailable.

Table 9. Proposed configuration defaults. Numeric batch and retrieval settings require measurement.

[TODO: select the annotation model and establish acceptable latency, cost, concurrency, microbatch delay, and retrieval budgets using replay measurements.]

12. Reliability, security, and auditability

12.1 Failure handling

Duplicate event delivery should have no effect after the first accepted event. Out-of-order arrivals should be held or correlated using identifiers and ordering metadata. A graph patch based on stale source versions should be rejected and rescheduled. A malformed annotation should create a visible unresolved state, not a guessed relationship.

A transport disconnect can leave a call's outcome unknown. Do not invent a ToolResult saying “failed” unless the runtime emits that result or error. A runtime-generated timeout report may be recorded with `producer_kind`=runtime, clearly distinguished from an external tool response. Canceled and permission-denied calls remain visible without implying external completion.

On restart, replay the append-only event log to restore execution state and load the latest graph snapshot. In-flight annotation requests can be retried as annotation jobs; their retries must not be confused with ToolCall `retry_of` links in the primary interaction.

12.2 Data and action boundaries

Reasoning, tool arguments, and retrieved content may contain confidential material. Store them under the same access boundary as the conversation. The proposed implementation should redact secrets from logs and exports, keep protected originals only where authorized, and prevent one user's graph from referencing another user's private sources.

Retain sufficient provenance to audit annotations without duplicating large source payloads across every node. Use immutable source references and hashes. A redacted export should indicate removed spans rather than replacing them with invented text. Document how deletion invalidates cached graphs and references.

[TODO: choose the persistence backend, retention/deletion policy, and allowed annotation-processing boundary for the intended deployment.]

12.3 Review and reproducibility

Store the schema version, prompt version, model identity, input hashes, and output annotations for each run. Human corrections create a later review revision. Keep the original annotation available for evaluation. A manual edit must distinguish a changed interpretation from a changed source message.

Observed execution links can still contain bad provider identifiers or adapter bugs; “recorded” is a statement about evidence origin, not an infallibility claim. Validation should check identifier consistency and record anomalies for review.

13. Validation and evaluation plan

13.1 Structural and lifecycle acceptance tests

Test ID	Fixture	Expected outcome
G01	Registry load	Exactly 11 node labels and 20 edge labels are available.
G02	Operational nodes in the legend and graph	ToolCall and ToolResult render as boxes.

Test ID	Fixture	Expected outcome
G03	Original semantic labels	All 14 names appear in colored label boxes with the approved family colors.
G04	Cyclic or self-referential edge	Validator rejects the patch.
G05	Two labels between the same nodes	Both valid relationships survive as distinct edges.
G06	No exposed plan before a call	ToolCall remains visible; no synthetic Planning node is created.
S01	Arguments split across several deltas	One provisional call updates until its request is complete.
S02	Parallel calls return in reverse order	Results attach through call IDs, not arrival position.
S03	Duplicate event delivery	One event projection and one active graph effect.
S04	Open tab during generation	Prefix backfill and live updates meet at the cursor without gaps or duplicates.
S05	End episode while tab is closed	Reconciliation is marked pending; no expensive job starts.
S06	Split a previously displayed text node	Atomic revision retires the old version and updates incident edges.
P01	Explicit citation to an unsupportive source	cites may exist; `grounded_in` and support are not inferred automatically.
P02	Evidence arrived after generation and was not available	No retrospective semantic dependency is committed.
P03	Same URL has snippet and full-text evidence	Distinct source representations and Context spans are retained.
P04	Technical retry with recorded lineage	Earlier call → later call `retry_of`; no automatic backtrack edge.
F01	Interrupted trace without an answer	Incomplete graph; no forced Conclusion.
F02	Tool request denied before dispatch	Requested/denied status; no false success or completion event.
F03	Empty search list	ToolResult exists; absence of hits is not promoted to a universal factual claim.
F04	Replay of a stored call	Visualization is restored without executing the operation.

Table 10. Proposed developer acceptance fixtures. These tests verify the design contract, not semantic annotation accuracy.

13.2 Semantic evaluation

Replay complete reasoning traces as simulated streams, keeping the source text identical across conditions. Compare a full-trace offline baseline, incremental annotation, incremental annotation with final reconciliation, and incremental annotation with bounded candidate retrieval. The paper's dataset can inform an evaluation if available under its applicable release conditions; this document does not claim to have run those experiments. [1, §§4–5; Appendix C]

Measure segmentation agreement, node-label accuracy or F1, labeled-edge precision/recall/F1, predecessor-candidate recall, and per-label performance. Report long-range edges separately. Automatic offline output is a comparison baseline, not human ground truth. Use a manually annotated subset for correctness claims and assess annotator agreement on the proposed new labels.

13.3 Interactive performance and stability

Measure the delay from a stable source unit to its first displayed annotation and to its final committed annotation. Record model requests, input/output tokens, queue lag, and actual execution overhead. Compare primary-response latency with annotation disabled and enabled.

Graph stability should measure node relabeling, segmentation changes, edge additions/removals among already displayed nodes, and layout displacement. Measure the tradeoff between a faster first view and subsequent revisions. Do not report a graph as more usable solely because it has lower latency; a later user study or task-based evaluation would be required for that claim.

13.4 Generalization limits

The paper evaluates mathematical, scientific, and argumentative traces, not the full range of multi-turn tool workflows. Web retrieval introduces source identity, availability, partial content, and operational status that its original evaluation does not cover. The extension therefore needs its own annotation study and adapter-level tests before research claims are made. [1, §4.2.1; Limitations; 2]

14. Implementation sequence and handoff

14.1 Suggested module boundaries

A `provider_adapter` module normalizes events and records capabilities. An `event_store` module persists raw input and cursors. An `execution_projector` creates calls, results, and extraction provenance. A segmenter manages source spans. An annotator performs role and relationship classification. A `candidate_index` retrieves eligible predecessors. A `graph_store` validates and versions changes. A visualization module renders snapshots and patches. An evaluator replays fixtures and scores annotations.

These are suggested responsibilities, not a prescribed framework or repository structure. The developer can implement them inside the existing chat application without adopting a particular vendor SDK, graph database, or orchestration library.

14.2 Delivery phases

Phase	Deliverable	Completion condition
1. Capture and replay	Normalized event log, context manifest, execution graph	Recorded calls/results reconstruct correctly, including errors, duplicates, and parallel returns.
2. Offline semantic baseline	Original ReasoningFlow labels plus the six new relationships	Validated graph export and manually checked sample annotations.
3. Live annotation	Stable-tail segmentation, microbatches, graph patches	Incremental behavior passes the lifecycle tests without blocking chat.
4. Lazy UI and caching	Approved legend, filtered views, backfill, cached reopen	No duplicate annotation on reopen; source and evidence inspection work.
5. Reconciliation and evaluation	Revision handling, offline/live comparison, metrics	Report measured accuracy, stability, latency, and cost with limitations.

Table 11. Proposed implementation sequence.

14.3 Open decisions

The three inline TODOs identify the required deployment decisions: the first provider contract in Section 5.2; annotation model and performance budgets in Section 11.3; and storage, retention, and processing boundaries in Section 12.2. Until those choices are made, use the typed internal contracts and mock/replay fixtures rather than assuming a provider-specific event layout.

Other choices in this specification are proposed defaults that the developer can test without changing the accepted taxonomy. A change to top-level labels, edge direction, or legend meaning requires a schema version change and an updated annotation guide. A change to batching or layout is a configuration or implementation revision unless it changes the annotation semantics.

15. Interpretation and limitations

The resulting graph represents the expressed reasoning structure plus recorded execution and evidence associations. The original ReasoningFlow paper finds a gap between text-level discourse relations and mechanistically measured dependencies; an extension that observes more tool events does not close that gap automatically. [1, §8]

A tool call can be observed without an exposed plan. A result can be returned without being passed to the model. A source can be cited without supporting the statement. A plausible grounding link can remain an annotator's inference. Keeping these distinctions visible is part of the data contract, not merely explanatory text in the user interface.

Streaming graph construction is feasible under the proposed earlier-to-later, revisable design. Its fidelity relative to offline annotation, its operating cost, and its usefulness to users remain evaluation questions. The implementation should preserve enough original evidence to answer those questions rather than treating a visually coherent graph as proof of correctness.

References and source record

[1] Jinu Lee, Shivam Agarwal, Amruta Parulekar, Siddarth Madala, Dilek Hakkani-Tür, and Julia Hockenmaier. 2026. *ReasoningFlow: Discourse Structures for Understanding LLM Reasoning Traces*. arXiv:2606.05402v1, 3 June 2026. Supplied PDF. Paper record. Primary sections used: §3, §4.2, §5.1, §§6–8, Appendix A Tables 3–4, Appendix C, Appendix I, and Figures 1 and 4. The supplied version is the source of record; this design does not claim to describe a later revision.

[2] Nathan Conklin and ChatGPT. 2026. *ReasoningFlow with tool calls and web search: design discussion and approved legend*. Current conversation, 11 October 2026. Design decisions and proposed extensions; not a published or empirically validated source. The approved final image is embedded in Section 2.

Companion specification: *ReasoningFlowWithToolCallAnnotationGuide.docx*. Appendix A extension defining the two new nodes and six added relationships, with Label, Description, Subtype, and Example tables, source-compatible label colors, and adjudication rules.